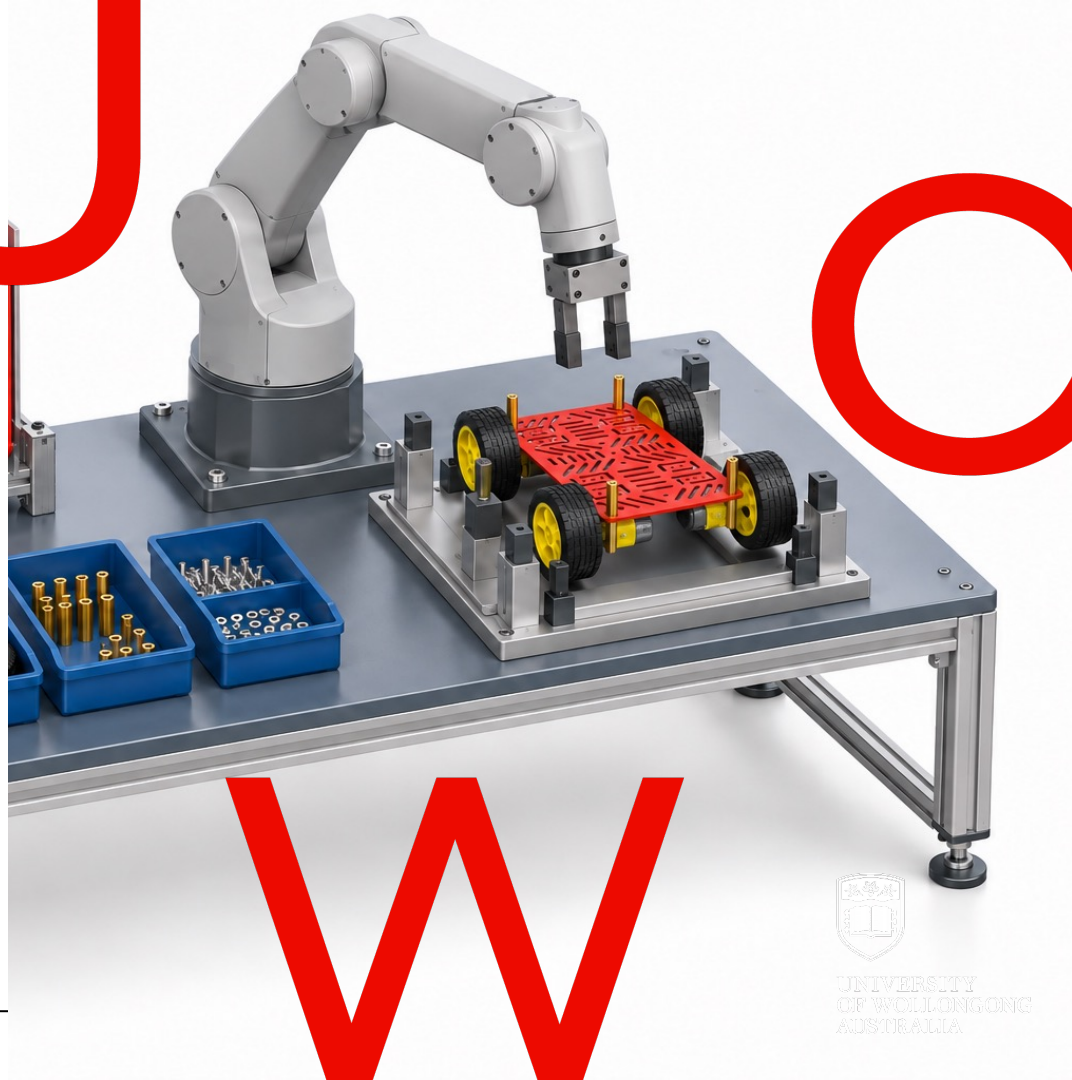
Robotics and Flexible Automation

Lecture 9
Robot Path Planning

Prof. Zengxi (Stephen) Pan

BJTU 2026



OPTIONAL MODULE

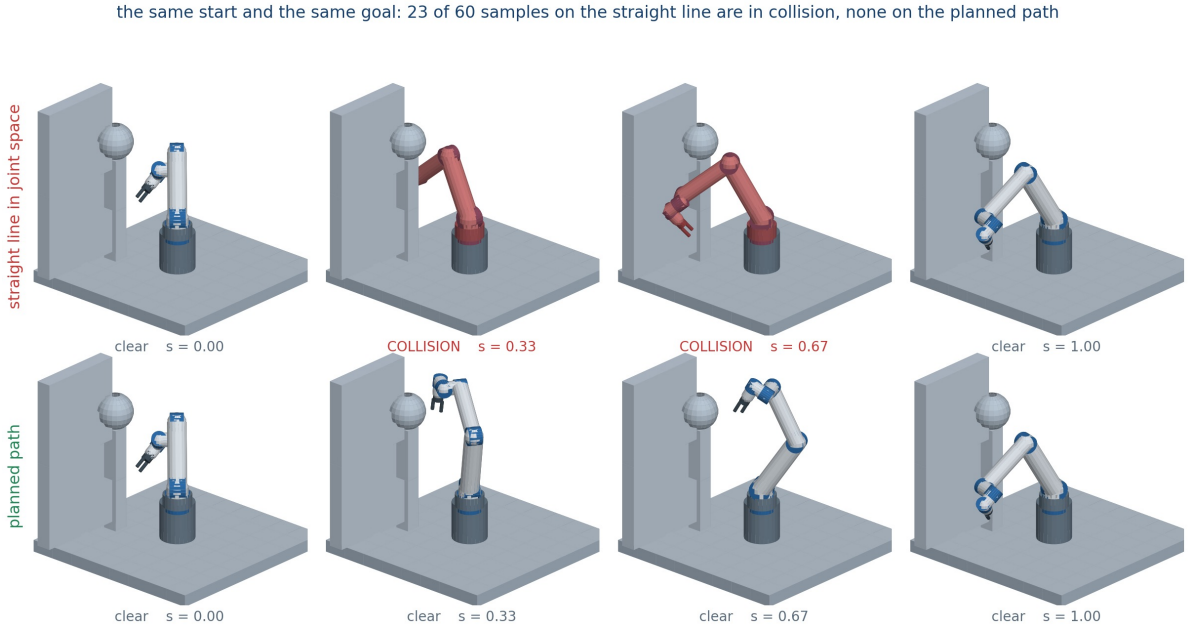
Robot Path Planning

Planning a collision-free route before trajectory generation

1. Problem definition, C-space and collision checking
2. Classical planners: graph search, PRM, RRT and optimisation
3. Corke MATLAB / Python tools and verification limits

The path-planning problem

OPTIONAL



what a planner must return

$$q(s) \in C_{free}, \quad s \in [0,1]$$

Start and goal pose

$$q(0) = q_{start}, q(1) = q_{goal}$$

both endpoints are valid — the motion between them is not

Path planning and trajectory planning

OPTIONAL

one decides where the robot passes, the other decides when it is there

Path planning — this lecture

In: start, goal, obstacles, joint limits.

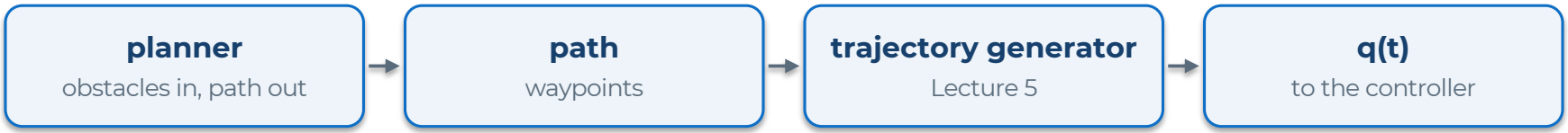
Out: configurations, or a continuous curve, that stay out of collision.

There is no time on it.

Trajectory planning — Lecture 5

In: a path, plus velocity and acceleration limits.

Out: q(t), with a duration, and the corners blended so it can be run at speed.



a planner's output is not yet a motion — slide 16 is where the two halves meet

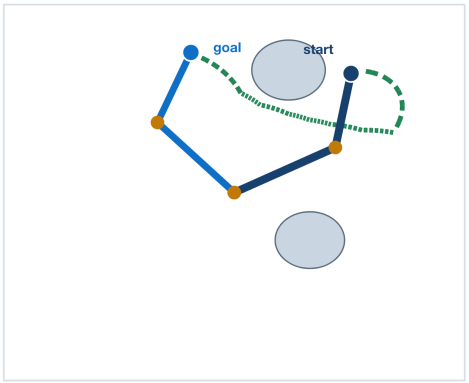
Workspace and configuration space

OPTIONAL

in the workspace the arm is a body with length; in configuration space it is one point

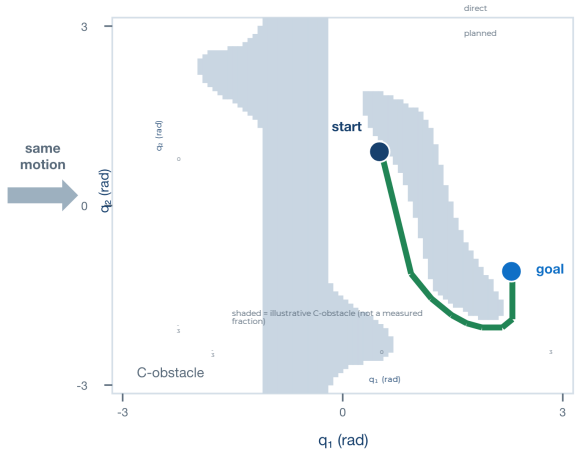
workspace: the arm is a body with length

Workspace: TCP path



configuration space: the arm is one point

Configuration space: joint path



a six-axis robot is one point

$$q \in R^6$$

and C-space splits in two

$$C = C_{free} \cup C_{obs}$$

a workspace obstacle maps to a C-obstacle whose shape has nothing in common with it

A motion of the robot is a curve in C-space; a collision-free motion is a curve that never enters Cobs.

The same motion appears as a TCP curve in the workspace and as a configuration curve in C-space.

For six-axis planning, Cobs is usually tested through a collision checker rather than explicitly enumerated.

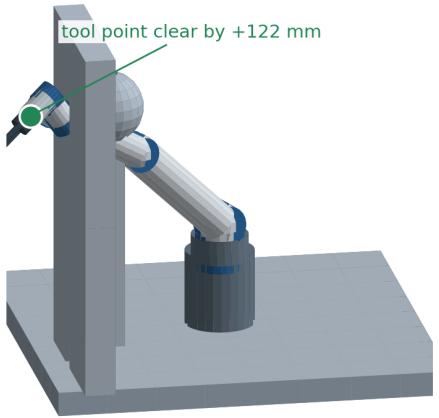
Collision checking

OPTIONAL

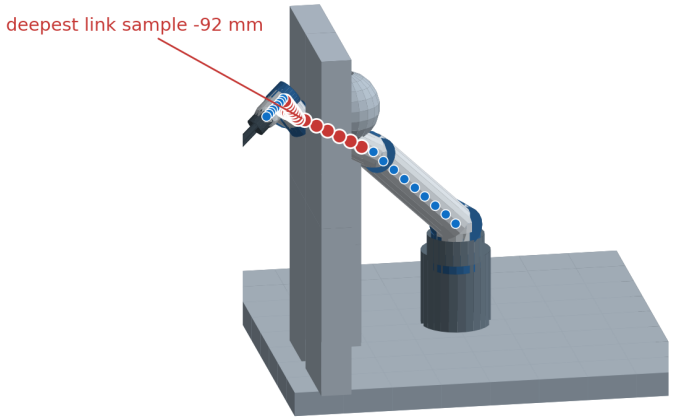
the inner loop — and where almost all of the planning time actually goes

a collision check is run on the whole robot, not on the tool point

the tool point alone is clear



the links are not



what has to be checked

- the robot against the environment
- the robot against itself
- the tool and workpiece against the fixture
- the joint limits

checking only the TCP is the commonest mistake

an edge is free only if every point on it is

$$free(a,b) \Leftrightarrow q(t) \in C_{free} \quad \forall\, t \in [0,1]$$

Why one planning query is difficult

OPTIONAL

even one start-to-goal query is six-dimensional and expensive to probe

High-dimensional space.

— Ten divisions per joint is a million cells for six.

Narrow passages.

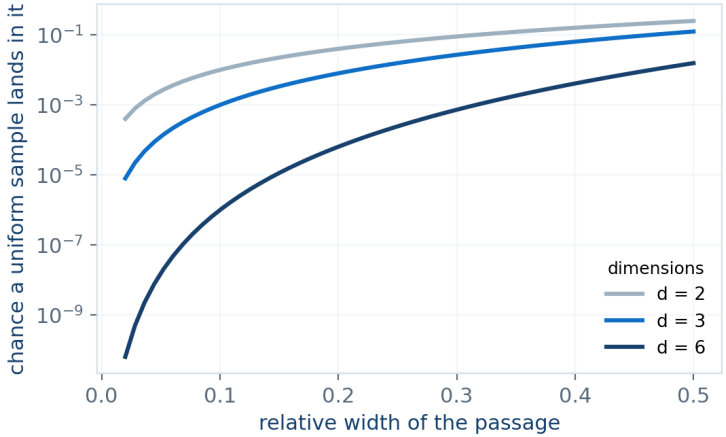
— A passage 10% of the range wide catches one uniform sample in a million at six dimensions.

Even one goal pose may have several IK solutions.

— Up to eight, and not all connect continuously from the same start.

Collision checking dominates the cost.

Short, clear and smooth pull against each other.



$$P(hit) = w^{\,d}$$

$$0.1^6 = 10^{-6}$$

the space cannot be enumerated, so it is probed

Why a practical welding pass is harder

OPTIONAL

100 partially constrained waypoints turn one query into a coupled sequence

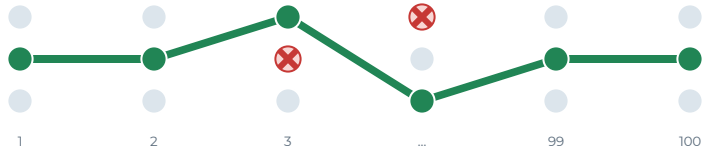
One failed point or transition can break the pass

Locked torch pose



collision

Torch angle allowed to vary



an alternative orientation restores a continuous route

How task freedom changes the scale

With K candidates at each waypoint:

about 100K candidate configurations
about $99K^2$ adjacent transition checks

fully specified pose	$K \approx 8$	8×10^2 nodes 6×10^3 checks
one relaxed angle, 10 samples	$K \approx 80$	8×10^3 nodes 6×10^5 checks
two relaxed angles, 10×10 samples	$K \approx 800$	8×10^4 nodes 6×10^7 checks

Illustrative upper bounds. Practical planners prune aggressively.

Relaxation changes both feasibility and search

A locked pose can be infeasible. A small torch-angle tolerance may create a valid pass, but the planner must coordinate that freedom across all 100 waypoints.

Three layers of motion planning

OPTIONAL

represent the possibilities, find a feasible route, then make it executable

1 REPRESENT

Build candidate structure

geometry: visibility graph, Voronoi
discretisation: grid, state lattice
sampling: PRM, RRT

graph, roadmap, or tree

2 SEARCH

Find a feasible route

Dijkstra or A* searches a graph
PRM searches its roadmap
RRT grows and searches together

collision-free path

3 REFINE

Improve and prepare

shortcut or optimise
verify collisions again
add timing and robot limits

executable trajectory

CROSS-CUTTING Potential fields guide local motion, costs, or sampling — Learning biases samples, heuristics, or costs

Example PRM samples and connects → A* or Dijkstra finds a route → shortcut and verify

Six families of planning tools

OPTIONAL

not mutually exclusive: practical planners combine them across the three layers

1 Geometric construction

visibility graph, Voronoi, cell decomposition

builds structure directly from obstacle geometry

2 Discretisation and graph search

grid, state lattice, Dijkstra, A*

search needs nodes and connections first

3 Potential and reactive methods

potential fields

acts locally or guides another planner

4 Sampling-based planning

PRM: sample, connect, then graph search

RRT: sample while growing a search tree

5 Trajectory optimisation

CHOMP, STOMP, TrajOpt

improves an initial path or trajectory

6 Learning-guided and hybrid

biases samples, heuristics, or costs

never replaces collision and validity checks

Graph search and A*

OPTIONAL

needed later: a roadmap is only useful once something searches it

A* expands nodes in order of

$$f(n) = g(n) + h(n)$$

g(n) — the cost already paid to reach n from the start.

h(n) — an estimate of the cost still to go.

If h never overestimates, A* returns the cheapest path in the graph, and expands fewer nodes than Dijkstra needs to get it.

Straight-line distance is admissible whenever the true cost is a path length, which is why it is the usual choice.

A* on a 46 x 46 grid: 657 cells expanded, 49 on the path



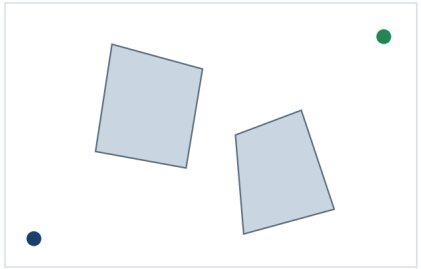
Algorithm illustration — not a timing or node-count benchmark.

Traditional geometric and potential methods

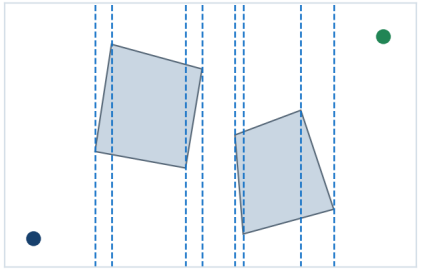
OPTIONAL

geometric constructions and local guidance: useful ideas, different scaling limits

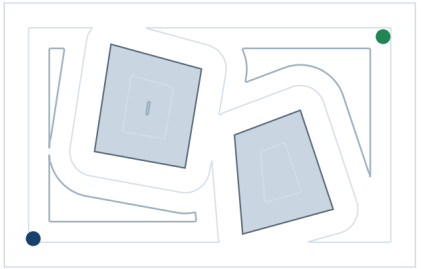
visibility graph



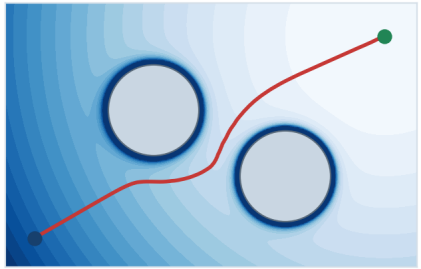
cell decomposition



maximum-clearance roadmap



artificial potential field



Visibility graph

shortest path among polygons — and it grazes every corner, so no clearance at all

Cell decomposition

exact and complete in the plane; the cell count explodes with dimension

Voronoi roadmap

maximum clearance by construction, at the cost of a longer path

Potential field

goal attracts, obstacles repel — and it stops in local minima

Illustrations only. High-dimensional extensions need suitable representations and collision checking.

Probabilistic roadmap

OPTIONAL

Build reusable connectivity, then search the roadmap for each query

1 Sample keep the collision-free ones

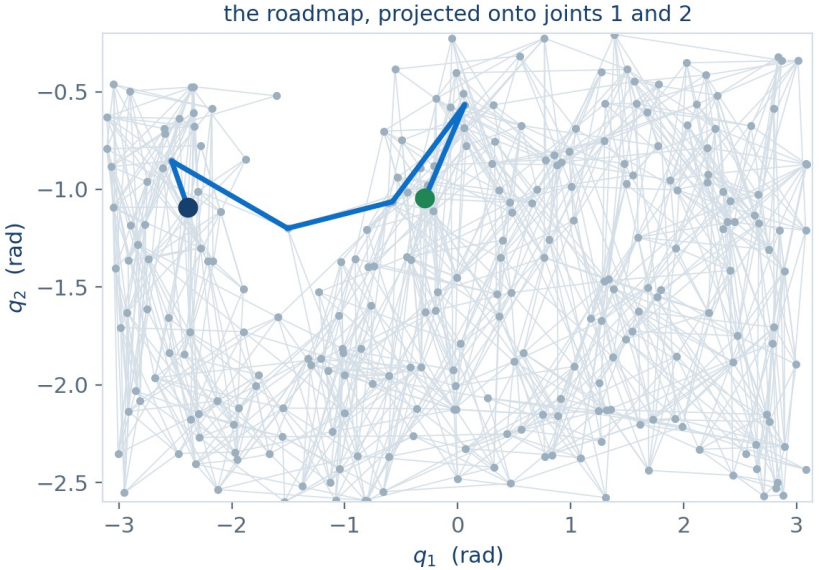
2 Connect each to its k nearest

3 Reject any edge not free along its length

4 Attach the start and the goal

5 Search the graph — Dijkstra or A*

Illustration: build once; search for each query.



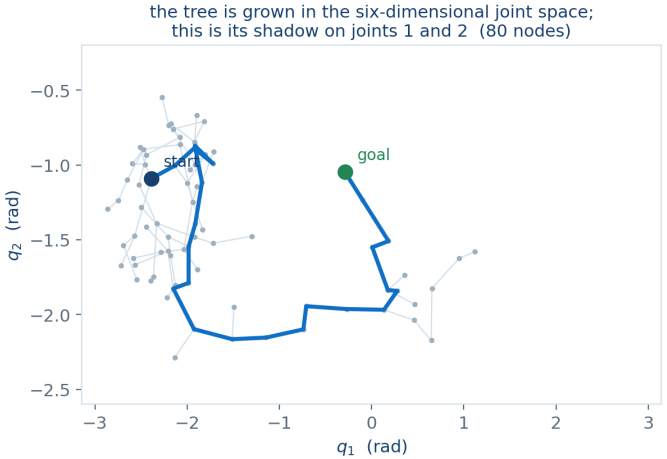
Joint-space projection; not a validated clearance map.

Rapidly exploring random tree

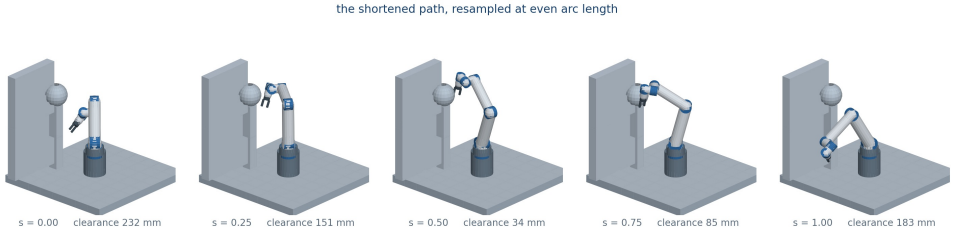
OPTIONAL

one query, one tree, grown from the start until it can reach the goal

1 sample — q_rand, goal-biased ▸ **2 nearest** — closest tree node ▸ **3 extend** — one step towards it ▸ **4 check** — the whole new edge ▸ **5 add** — if the edge is free ▸ **6 stop** — goal reachable



Joint-space projection; not a 6D collision proof.



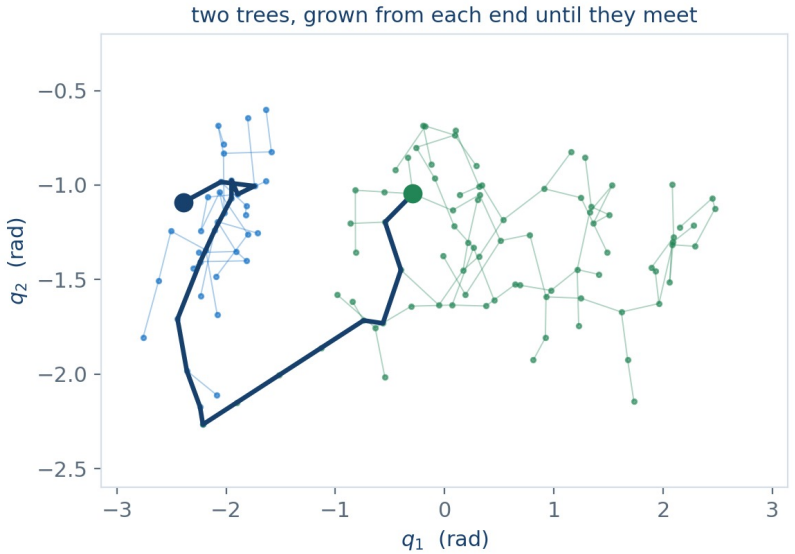
Illustrative RRT tree and arm poses — no verified run statistics.

Moderate goal bias directs growth. Too much reduces exploration. Check every full-robot edge.

RRT-Connect and RRT*

OPTIONAL

the two variants an engineer actually meets



Algorithm illustration — two trees grow towards a connection.

RRT-Connect

A tree from the start and a tree from the goal; after each extension each tries to reach the other.

Often faster than basic RRT for a single feasibility query. Validate every new connection.

RRT*

Rewires the tree whenever a new node offers its neighbours a cheaper route.

The path keeps improving the longer it runs — asymptotically optimal — for considerably more computation.

From a planned path to an executable motion

OPTIONAL

what a planner returns is not yet something a robot can run

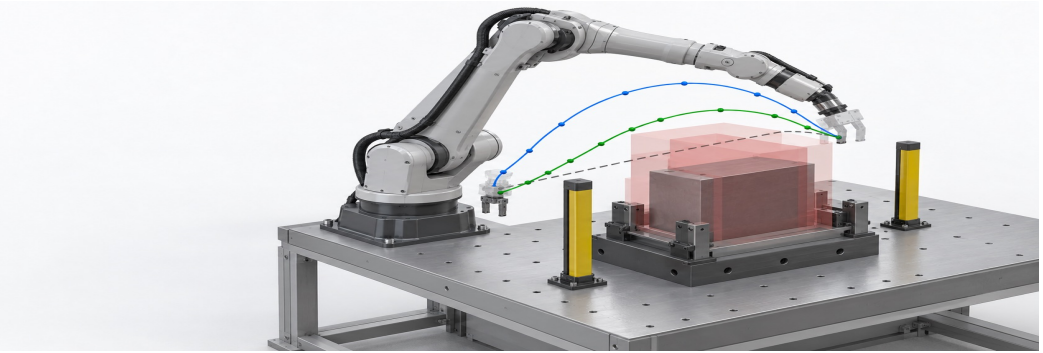


Illustration only: a raw sampling path can contain redundant waypoints and sharp corners. Shorten, smooth and assign time, then recheck the edited motion.

a path that has been shortened, resampled or edited by hand has not been checked

1 **Shorten** random shortcuts that stay free

2 **Smooth** blend the corners

3 **Interpolate** resample for the controller

4 **Time** apply the limits — Lecture 5

5 **Verify again** the edited path is unchecked

the last step is not optional

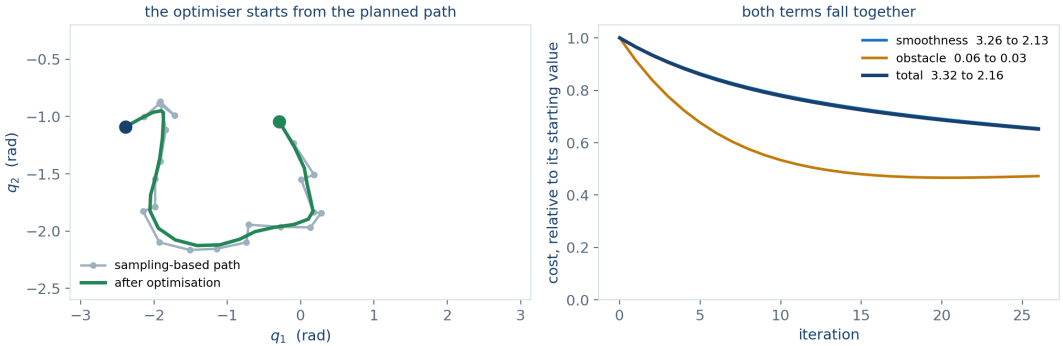
Optimization-based planning

OPTIONAL

improve a path you already have, by writing down what a good one would mean

one cost function, three competing terms

$$J(q) = J_{smooth}(q) + \lambda J_{collision}(q) + J_{constraints}(q)$$



Illustrative cost trends, not a measured improvement or clearance result.

CHOMP · STOMP · TrajOpt
CHOMP: covariant gradient updates.
STOMP: sampled perturbations.
TrajOpt: sequential convex subproblems.
Different formulations; local optimisation needs an initial route.

so: sample first, optimise second

Choosing a planner

OPTIONAL

there is no best planner — there is a best starting point for each situation

situation	start with	why
Repeated queries, fixed cell	PRM	the build cost is paid once
One start-goal query	RRT-Connect	nothing to build first
Feasible but too long	shortcutting, or RRT*	shortcutting is usually cheaper
Needs margin or smoothness	CHOMP, STOMP, TrajOpt	writes the requirement down as a cost
Strict LIN / PTP motion	the controller's own planner	the motion type is guaranteed
The environment changes	replanning	a stored path is invalid once the cell moves

in a real cell: plan once at commissioning, verify it, store it, and run the stored path

MATLAB / Corke RTB10: candidate motion

OPTIONAL

Generate and inspect a joint path; collision planning is a separate task

```
% Peter Corke RTB10, not MathWorks
mdl_puma560
qA = p560.qz;
qB = [0 0.5 -0.5 0 0.5 0];
t = linspace(0, 2, 101);
[q, qd, qdd] = jtraj(qA, qB, t);
T = p560.fkine(q);
p560.plot(q);

% Joint-limit check at each sample
lo = q < p560.qlim(:,1).';
hi = q > p560.qlim(:,2).';
withinLimits = ~any(lo | hi, 2);

% Candidate motion, not a
% collision-checked assembly path.
```

Corke provides the arm model, kinematics and trajectory samples.

jtraj does not avoid obstacles. Check every link, the tool and the carried part with a separate geometric checker.

The supplied Puma model is a teaching example, not the project robot.

✗ Robotics System Toolbox
NOT USED IN THIS COURSE

Python / Corke: model and verification

OPTIONAL

Same modelling route as MATLAB, with Python-specific APIs

Corke robot model

Use DH models for the course examples. Keep base, tool, joint signs and units consistent.

Joint trajectory

rtb.jtraj returns q, qd and qdd. Forward kinematics evaluates the resulting tool poses.

Collision planning

An arm needs a whole-body geometric checker. Mobile-robot PRM/RRT examples are not six-axis arm planners.

Kinematics and a plotted trajectory do not establish collision-free assembly.

```
import roboticstoolbox as rtb
robot = rtb.models.DH.Puma560()
qB = [0, 0.5, -0.5, 0, 0.5, 0]
traj = rtb.jtraj(robot.qz, qB, 101)
poses = robot.fkine(traj.q)
```

No MathWorks robotics API is required. Real project geometry still needs independent collision verification.

Optional motion-verification exercise

OPTIONAL

Apply Corke kinematics and trajectory generation to a candidate route

1. **Model a robot in Corke MATLAB or Python**
2. **Choose two feasible joint configurations**
3. **Generate a candidate joint trajectory**
4. **Inspect limits, tool path and singularity proximity**
5. **Compare direct motion with a via-point route**
6. **Explain what a separate collision checker must test**

Discuss

Model and units

Path versus timing

Joint limits

Robot, tool and part clearance

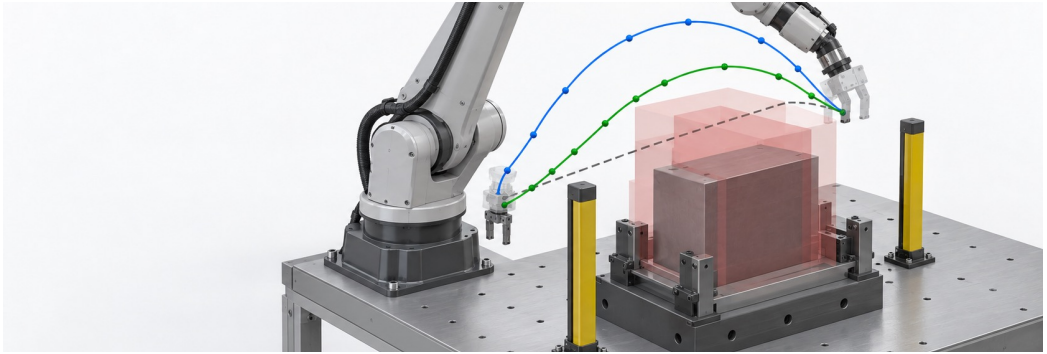
What remains unverified

A Corke plot supports kinematic reasoning. It does not certify a collision-free robot cell.

AI-assisted path planning

OPTIONAL

AI can guide candidates; geometry, kinematics and collision checks still decide validity



AI ranks samples or proposes seeds from scene data.

Classical collision checks and joint limits certify the final motion.

A language or vision model can suggest a route, but it cannot replace the collision model

1 Perceive build or update the collision world

2 Guide bias sampling or propose useful seeds

3 Validate test each robot state and path segment

4 Optimise improve smoothness, clearance and length

5 Replan react when vision detects a changed scene

validation remains mandatory


UNIVERSITY
OF WOLLONGONG
AUSTRALIA
Stand
for purpose
THANK YOU!